

CITY UNIVERSITY MALAYSIA
Cyberjaya Campus — Faculty of Information Technology
BIT2083 Fundamental of Computational Thinking: Python
Final Project Report (40%)

15-Minute Neighborhood Score

A Console Application (and Companion Website) for
SDG 11: Sustainable Cities and Communities

Full Name	[Your Full Name]
Student ID	[Your Student ID]
Class Code	[Your Class Code]
Program	[Your Program]
GitHub Repository	github.com/DevFadul/15-minute-neighborhood-score
Live Website	[paste your Render URL here once deployed]

August 2026

Note: replace the bracketed placeholders above with your own details (and each group member's) before submission, per the assignment instructions. NRIC/passport numbers are not included in this document. Enter that information directly wherever your submission process requires it.

Contents

1. Introduction & SDG Background
2. Problem Statement
3. System Objectives
4. Application Design
5. Implementation Details
6. Testing & Sample Outputs
7. Discussion & Limitations
8. Conclusion

1. Introduction & SDG Background

Most people don't think twice about driving to buy groceries or see a doctor. But that habit adds up: more traffic, more pollution, and neighborhoods that stop working for anyone who can't drive, like kids, older adults, or families who can't afford a car.

This project is about United Nations Sustainable Development Goal 11 (SDG 11): **Sustainable Cities and Communities**. It's the UN's goal of making cities safer, greener, and easier to live in without needing a car for every errand. Two parts of that goal matter most here:

- **Target 11.2:** everyone should have access to safe, affordable transport.
- **Target 11.7:** everyone should have access to green space and public places.

Both come down to one question: *how close are the things you need to your front door?*

City planners have a name for a neighborhood that answers “close enough”: a **15-minute city**. A resident should be able to walk to most of their daily needs (food, healthcare, school, transit, parks, everyday shops) in about 15 minutes.

This project turns that idea into something anyone can use. It's called the **15-Minute Neighborhood Score**, and it now exists in two forms: a Python console program, and a web app added later with a 3D map. Both score how walkable a neighborhood is and say exactly what's missing.

2. Problem Statement

It's hard to know if a neighborhood is walkable just by living in it. You might know your nearest grocery store is close, but is the rest of the neighborhood just as good? And if you're choosing between two apartments, how do you compare them fairly instead of guessing?

The tools that already exist fall into two camps: GIS software built for city planners, too complex for a renter to bother with, or websites that need a live internet connection, an account, and a paid mapping API.

So the problem this project solves is simple. Let anyone, using a basic computer or a web browser, describe how long it takes to walk to their nearest grocery store, clinic, school, bus stop, park, and shop, and turn that into one clear, comparable score with specific tips on what to improve. That helps people make a more informed, less car-dependent choice about where to live.

3. System Objectives

The 15-Minute Neighborhood Score is built to:

- Let a user record a “neighborhood assessment”: how many minutes it takes to walk to six essential categories of places.
- Turn that into a single score from 0 to 100, with an easy-to-understand rating: Excellent, Good, Fair, or Poor.

- Save every assessment, so a user's history is still there the next time they open the program.
- Point out exactly which categories are weakest, with specific, realistic tips for each one.
- Let a user compare two saved assessments side by side, to help decide between two homes.
- Never crash on bad input: always explain the mistake in plain language and ask again.
- Show computational thinking (breaking the problem into small, single-purpose functions) throughout the code.
- **(Added later)** Offer all of the above through an easy, visual website, including automatically estimating walk times for any real address, without the user needing to measure anything themselves.

4. Application Design

This project has two versions that share the same underlying code:

- **The console app:** the original version. Runs in a terminal window with a text menu.
- **The web app:** added afterward. Runs in a normal browser, with a 3D globe on the homepage, a visual dashboard, and a first-person 3D view where you can walk around a simplified version of your neighborhood.

Both versions call the same Python functions to do the scoring math. That code lives in one file, `neighborhood_score.py`, so the console app and the website always agree on the score for the same data.

4.1 How the Console App Is Organized

The console app is written as two files. `neighborhood_score.py` holds the data and the math: the category weights, the walking-time table, and a set of plain functions like `calculate_total_score()` and `classify_rating()`.

`main.py` is the program itself: the menu, all the screen output, and a small set of input functions that keep re-asking until the user gives a valid answer, e.g. `get_minutes()`, which loops until it gets a real number between 0 and 180.

There are no classes. An assessment is just a dictionary with keys like `nickname` and `category_times`, saved straight to a JSON file. Every function either takes that dictionary in and returns a value (the scoring functions), or reads and writes it directly (the storage functions).

Figure 1 shows how the program runs, step by step: starting up, showing the menu, and following one option (Create New Assessment) all the way through.

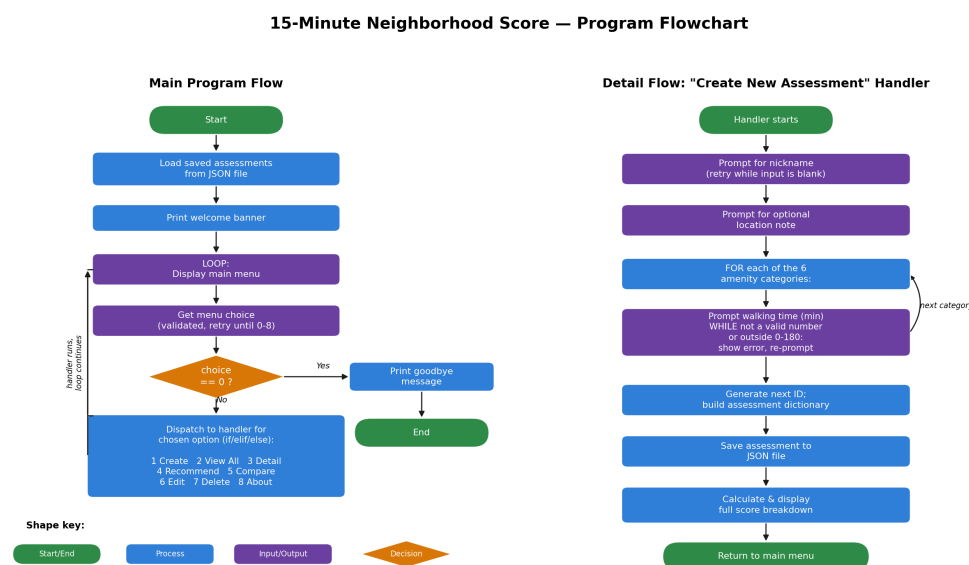


Figure 1. Program flowchart — main loop (left) and Create New Assessment detail flow (right).

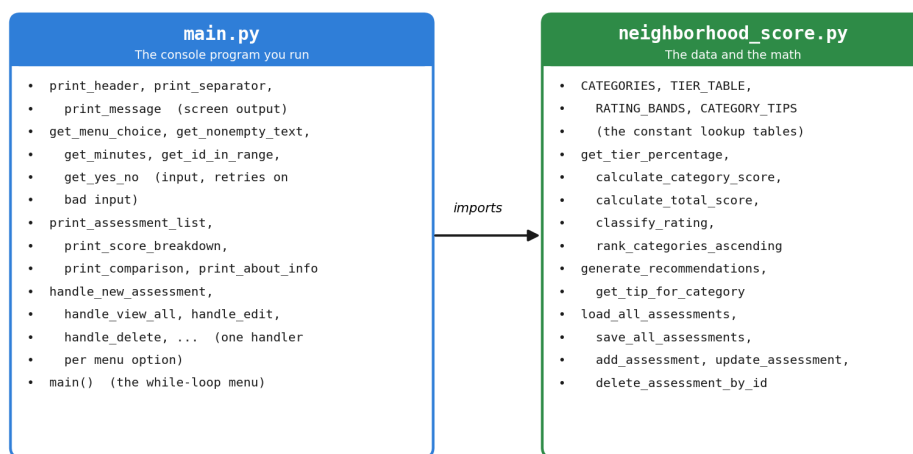
4.2 Program Structure

Figure 2 lays out what lives in each of the two files, and the one-way relationship between them: `main.py` imports `neighborhood_score.py`, never the other way around.

Splitting the program this way, one file for what the data is and how to score it, one file for the program the user runs, is enough separation to keep the code readable without needing classes or a deeper folder structure.

15-Minute Neighborhood Score — Program Structure

Two files. No classes -- everything is a plain function working on dictionaries, and data is passed between functions as arguments.



An assessment is just a dictionary -- e.g. `{"id": 1, "nickname": "Current Apartment", "category_times": {"grocery": 4, ...}}` -- saved straight to and read straight from JSON.

Figure 2. What each file contains, and how they connect.

4.3 How the Web App Is Built

The website reuses the exact functions from `neighborhood_score.py` shown above. It wraps a friendlier interface around them, built with a Python web framework called Flask. Its main screens are:

- A rotating 3D globe on the homepage (Figure 3), where a user can type any real address.
- A real address lookup: the app looks up that address using free map data from OpenStreetMap, automatically finds the closest real grocery store, clinic, school, bus stop, park, and shop, and estimates realistic walking times without any manual measuring.
- A dashboard (Figure 4) listing every saved assessment as a card, each with a small map-style preview and a walkability “ring.”
- A first-person, 3D “walkthrough” screen where a user can literally walk around a simplified 3D neighborhood and watch the distance to each place update live.
- Simple forms, a detail page, and a compare page, all styled to look like pages from an old paper map or atlas.

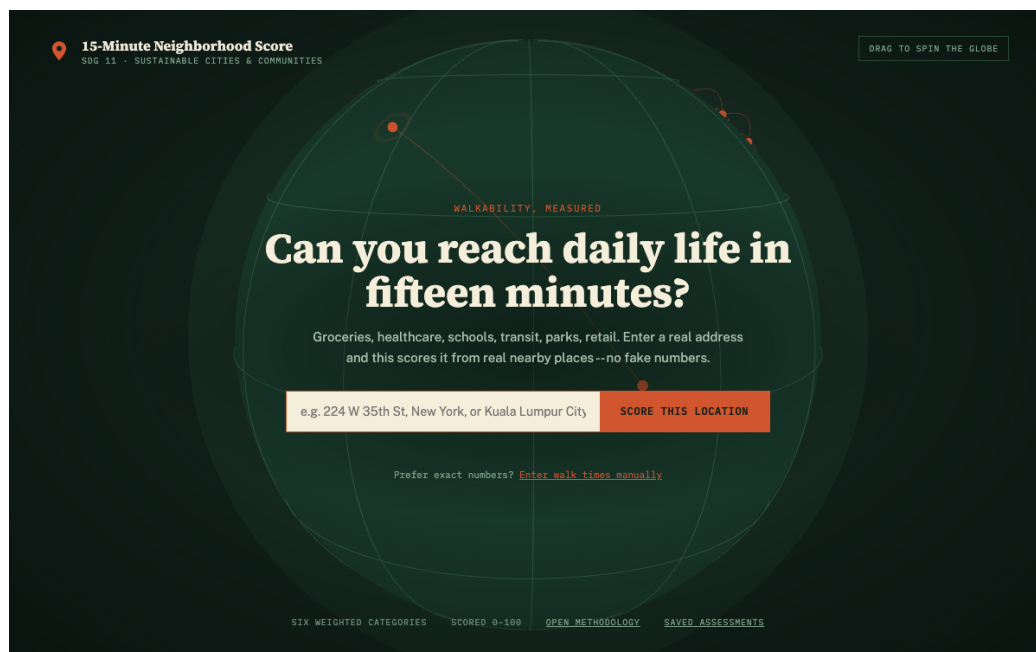


Figure 3. The web app's homepage — a rotating 3D globe with a real-address search box.

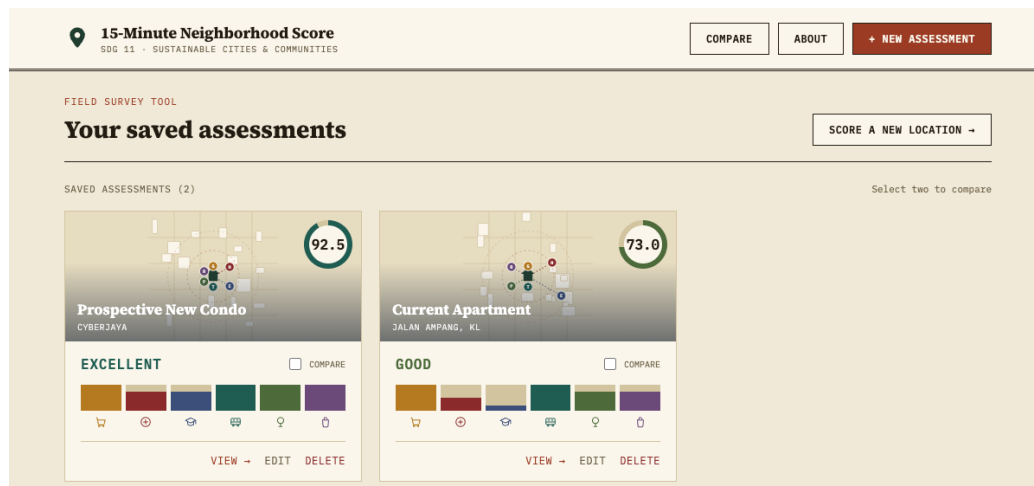


Figure 4. The web app's dashboard, listing saved assessments as cards.

5. Implementation Details

5.1 The Scoring Formula

Each of the six categories (Grocery, Healthcare, Education, Transit, Parks, Retail) is worth a set number of points out of 100, its “weight.” How many of those points a neighborhood earns depends on how close that category is:

Walking time	Points earned
0–5 minutes	100% of that category's points
6–10 minutes	75%
11–15 minutes	50%
16–25 minutes	20%
More than 25 minutes	0%

Add up the points earned across all six categories, and the result is a single score from 0 to 100. That score is then turned into a simple rating: **Excellent** (85 or above), **Good** (70–84), **Fair** (50–69), or **Poor** (below 50).

5.2 Estimating Real Addresses (Web App Only)

When a user types a real address into the website, the app does three things automatically:

- Turns the address into map coordinates (latitude and longitude), using a free service called **Nominatim**.
- Searches nearby, real-world map data (from a free service called **Overpass**) for the closest grocery store, clinic, school, bus stop, park, and shop.
- Estimates the walking time to each one, based on the straight-line distance and an average walking speed.

This isn't a perfect replacement for walking the real route. A straight line “as the crow flies” is usually a bit shorter than the real path along streets and sidewalks, but it gives a useful, realistic estimate in seconds for any real address, without the user needing to measure anything by hand.

5.3 Never Crashing on Bad Input

`main.py` has a small set of input functions, one per kind of question: `get_menu_choice()`, `get_nonempty_text()`, `get_minutes()`, and so on. Each one is just a while loop: ask, check the answer, and either return it or print an error and ask again.

If someone types letters instead of a number, a negative number, or an unrealistically large travel time, the program doesn't crash. It explains what was wrong and asks again. Saved data gets the same care: if the data file on disk is corrupted, the program renames it, warns the user, and starts with an empty history instead of crashing on startup.

6. Testing & Sample Outputs

6.1 How It Was Tested

The console app was tested by running it directly and feeding it real input, including a lot of deliberately wrong input, and checking it always responded correctly instead of crashing. This included:

- Starting the program with no saved data yet, and with a corrupted data file.
- Typing invalid menu choices, invalid numbers, negative numbers, and unrealistically large numbers.
- Checking the score and rating at every boundary of the walking-time table (e.g., exactly 5 minutes vs. 6 minutes).
- Getting recommendations for both a perfect score and a very low score.
- Comparing two different assessments, and trying (and correctly failing) to compare an assessment against itself.
- Editing and deleting assessments, then confirming the changes were still there after closing and reopening the program.

All of these worked as expected. The excerpt below is real program output, not a mockup:

```
Enter a nickname for this neighborhood: Current Apartment
Enter an optional location note (or press Enter to skip): Jalan Ampang, KL

Grocery / Fresh Food Access (minutes): 4
Healthcare / Pharmacy (minutes): 12
Education / Childcare (minutes): 20

[OK] Assessment #1 "Current Apartment" saved.
-----
SCORE BREAKDOWN: Current Apartment (ID 1)
-----
Category                Minutes   Tier    Points
-----
Grocery / Fresh Food Access    4.0    100%    20.0 / 20
Healthcare / Pharmacy          12.0    50%     7.5 / 15
Education / Childcare          20.0    20%     3.0 / 15
-----
TOTAL SCORE: 73.0 / 100  ->  Rating: GOOD
```

Excerpt from tests/manual_test_transcript.txt (shortened here for space).

The web app was tested by hand in a real browser: creating, editing, and deleting assessments, watching the live score preview update while filling in the form, comparing two assessments, walking through the 3D neighborhood view, and searching several real addresses (from a dense city center to a remote location) to confirm the automatic walk-time estimate behaves sensibly in both cases.

6.2 A Real Bug Found During Testing

Testing found a real bug, not just confirmed things worked. In an early version of the “Compare Two Assessments” screen, a long neighborhood nickname could overflow its column in the results table and run into the next column, because Python doesn't automatically shorten long text to fit a fixed width. The fix added a small helper function that safely shortens any text that's too long for its space, and applied it everywhere a name gets displayed, not just where the bug was first noticed.

7. Discussion & Limitations

By turning “is my neighborhood walkable?” into one clear, comparable number, this project gives people a practical tool for making a more sustainable choice about where to live, which is what SDG 11 is about. The comparison feature addresses a real decision people face (choosing between two homes), and the recommendations turn a plain number into something actionable.

A few honest limitations remain:

- The automatic address lookup (in the web app) estimates walking time using a straight line, not the real walking route along actual streets, so it's a helpful estimate, not an exact measurement.
- Every user gets the same category weights, even though different people have different priorities: a family with young children might care about Education far more than a retired couple would.
- The score measures whether something is nearby in time, not whether it's safe, pleasant, or well-lit to walk to.
- Data is stored in one file (or, once the site is deployed online, in the hosting service's storage) rather than a proper shared database, which is enough for one person's own use but not built for many people using it as a shared service at once.

Worth noting: one limitation from the original version of this project (“no mapping integration, so the user has to estimate their own walking times”) has since been solved. The web app now does exactly that automatically, for any real address, using free map data.

8. Conclusion

This project set out to use Python and computational thinking to solve a real, everyday sustainability problem under SDG 11: helping people understand and compare how walkable their neighborhood is. The result is two working tools that share the same underlying code: a tested, menu-driven console program, and a full interactive website, complete with a 3D globe, real address lookups, and a 3D neighborhood you can walk around, that turns a vague feeling of “this area seems walkable” into a clear, useful, comparable score.